ИТМО

Алгоритмы и структуры данных

Лекция 5

Статичные структуры данных

Еще раз о том, почему C++ а не python

ИТМО

```python
def do_smt(array: list[int]):
    array_unique = list(set(array))

    dict_count = dict(map(lambda v: (v, array.count(v)), array_unique))

    counts_list_unique = list(
        set(
            list(dict_count.values())
        )
    )

    ...
```



Не надо так…

Внутри то всё равно цикл...

ИТМО

Идем в исходники Python: https://github.com/python/cpython/blob/main/Objects/setobject.c

```c
static PyObject *
set_new(PyTypeObject *type, PyObject *args, PyObject *kwds)
{
    return make_new_set(type, NULL);
}
```

```c
static PyObject *
make_new_set(PyTypeObject *type, PyObject *iterable)
{
    assert(PyType_Check(type));
    PySetObject *so;

    so = (PySetObject *)type->tp_alloc(type, 0);
    if (so == NULL)
        return NULL;
```

```c
static int
set_update_iterable_lock_held(PySetObject *so, PyObject *other)
{
    _Py_CRITICAL_SECTION_ASSERT_OBJECT_LOCKED(so);

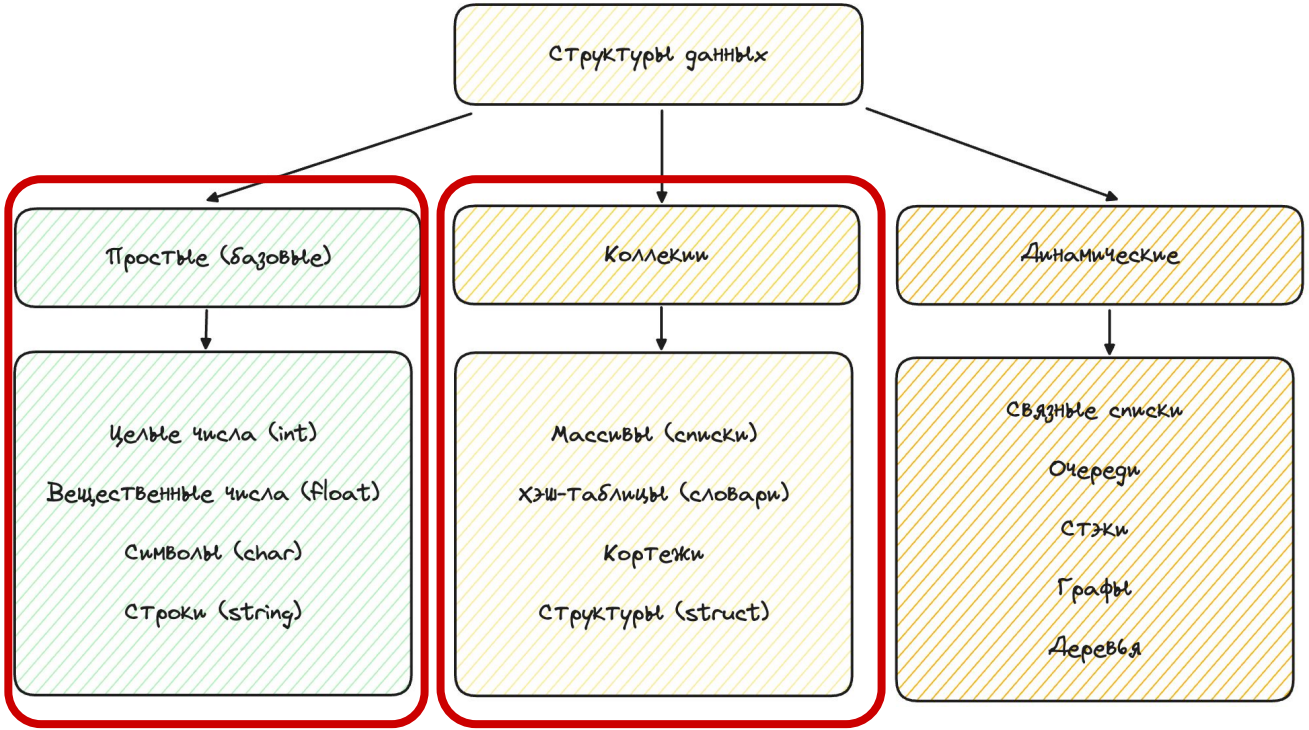
    PyObject *it = PyObject_GetIter(other);
    if (it == NULL) {
        return -1;
    }

    PyObject *key;
    while ((key = PyIter_Next(it)) != NULL) {
        if (set_add_key(so, key)) {
            Py_DECREF(it);
            Py_DECREF(key);
            return -1;
        }
        Py_DECREF(key);
    }
    Py_DECREF(it);
    if (PyErr_Occurred())
        return -1;
    return 0;
}
```

Видите цикл?

А он есть

О чем лекция


Структуры данных
Простые (базовые)
Коллекции
Динамические
Целые числа (int)
Вещественные числа (float)
Символы (char)
Строки (string)
Массивы (списки)
Хэш-таблицы (словари)
Кортежи
Структуры (struct)
Связные списки
Очереди
Стэки
Графы
Деревья

А теперь к типам (структурам)



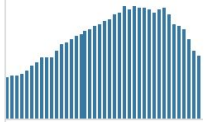
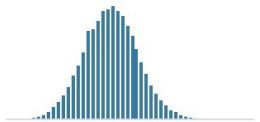
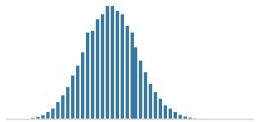
Почему важно понимать память

ИТМО

Chess Games

64 | New Notebook | Download (2 GB)

Data Card | Code (6) | Discussion (1) | Suggestions (0)

'CTime time	# WhiteElo White's ELO	# BlackElo Black's ELO	# WhiteRatingDiff White's rating points difference after the game	# BlackRatingDiff Black's rating points difference after the game
 -02-15 2021-02-16	 737 3110	 728 3108	 -595 673	 -653 664
0:01	1901	1896	11.0	-11.0
0:01	1641	1627	-11.0	12.0
0:02	1647	1688	13.0	-13.0

int64

или

int16

Если не следить за типами: **7.21 ГБ**

Если следить за типами: **3.90 ГБ**

https://www.kaggle.com/datasets/arevel/chess-games

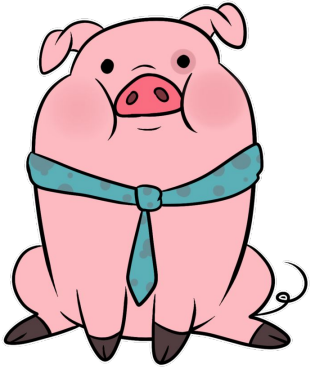
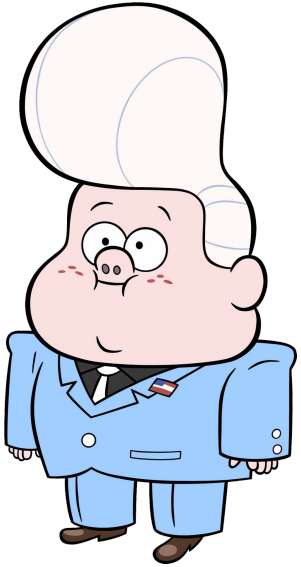
Перечислим базовые типы/структуры

ИТМО

Что помните?

Перечислим базовые типы/структуры

ИТМО

bool	int	float	char	string
				
простой, маленький, но иногда неожиданно решающий	понятный, надёжный, без лишней сложности	как будто int, но с дополнительной долей хаоса	маленький, один, но способен доставить много проблем	сложная, полный хаос, но без нее никак

Ну и коллекции

ИТМО

Структуры



внутри куча разных полей: знания, оружие, дневники, измерения, аномалии

Списки



можно построить мега-гнома через простые связи

Массивы



много одинаковых объектов одного типа
dippers[0], dippers[1]

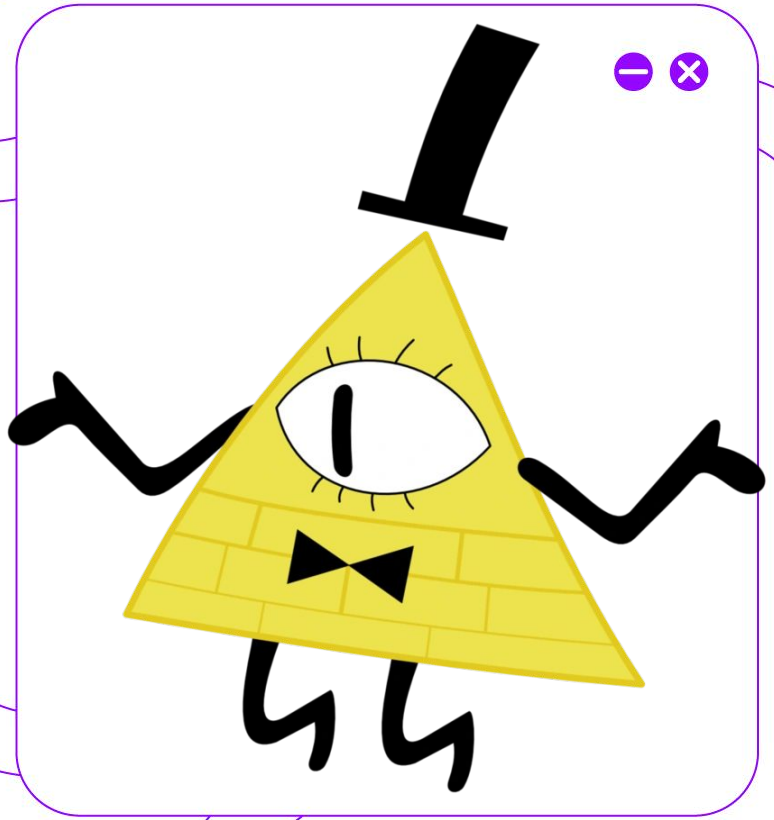
Хэш-таблицы



быстро, но внутри полный бардак

Указатели

пожалуй, идеальный pointer: сам находится в одном месте, но получает доступ к чужому разуму/телу/памяти



Начнем с простого – bool

ИТМО

```
printf("Size of true: %ld bytes\n", sizeof(true));  // -> ? bytes
printf("Size of false: %ld bytes\n", sizeof(false));  // -> ? bytes

printf("Size of _Bool: %ld bytes\n", sizeof(_Bool));  // -> ? bytes
printf("Size of bool: %ld bytes\n", sizeof(bool));  // -> ? bytes

bool true_ = true, false_ = false;
printf("Size of var true: %ld bytes\n", sizeof(true_));  // -> ? bytes
printf("Size of var false: %ld bytes\n", sizeof(false_));  // -> ? bytes

bool arr[10];
printf("Size of bool arr[10]: %ld bytes\n", sizeof(arr));  // -> ? bytes

bool* arr_ = malloc(10 * sizeof(bool));
printf("Size of bool* arr: %ld bytes\n", sizeof(arr_));  // -> ? bytes
```

Начнем с простого – bool

ИТМО

```
printf("Size of true: %ld bytes\n", sizeof(true));  // -> 4 bytes
printf("Size of false: %ld bytes\n", sizeof(false));  // -> 4 bytes

printf("Size of _Bool: %ld bytes\n", sizeof(_Bool));  // -> 1 bytes
printf("Size of bool: %ld bytes\n", sizeof(bool));  // -> 1 bytes

bool true_ = true, false_ = false;
printf("Size of var true: %ld bytes\n", sizeof(true_));  // -> 1 bytes
printf("Size of var false: %ld bytes\n", sizeof(false_));  // -> 1 bytes

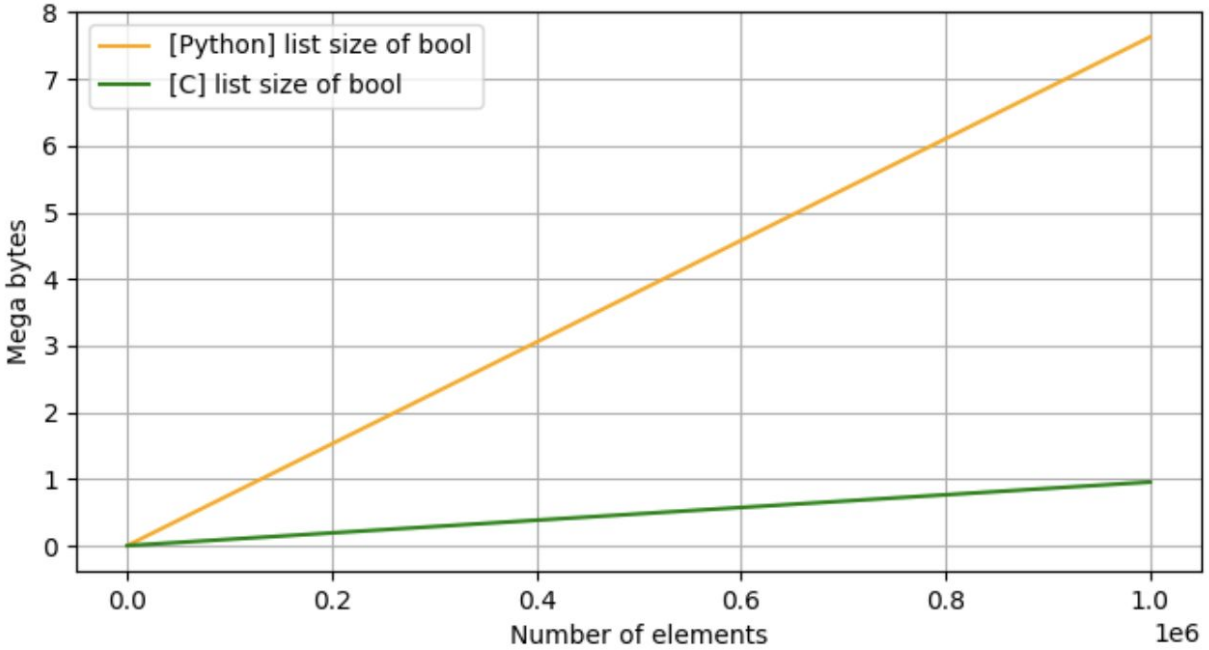
bool arr[10];
printf("Size of bool arr[10]: %ld bytes\n", sizeof(arr));  // -> 10 bytes

bool* arr_ = malloc(10 * sizeof(bool));
printf("Size of bool* arr: %ld bytes\n", sizeof(arr_));  // -> 8 bytes
```

Boolean в Python

ИТМО

```
print(f'True: {sys.getsizeof(True)} bytes')  # 28 bytes
print(f'False: {sys.getsizeof(True)} bytes')  # 28 bytes
```



Один плюс…

```
id(True), id([True][0])
(4302557144, 4302557144)
```

Integer в C

ИТМО

```c
printf("Size of 1: %ld bytes\n", sizeof(1));  // -> ? bytes
printf("Size of 2 147 483 647: %ld bytes\n", sizeof(2147483647));  // -> ? bytes
printf("Size of 2 147 483 648: %ld bytes\n", sizeof(2147483648));  // -> ? bytes

printf("Size of int8_t: %ld bytes\n", sizeof(int8_t));  // -> ? bytes
printf("Size of uint8_t: %ld bytes\n", sizeof(uint8_t));  // -> ? bytes
printf("Size of char: %ld bytes\n", sizeof(char));  // -> ? bytes
printf("Size of int s: %ld bytes\n", sizeof(short int));  // -> ? bytes
printf("Size of int: %ld bytes\n", sizeof(int));  // -> ? bytes
printf("Size of int l: %ld bytes\n", sizeof(long int));  // -> ? bytes
printf("Size of int ll: %ld bytes\n", sizeof(long long));  // -> ? bytes

int arr[10];
printf("Size of arr[10]: %ld bytes\n", sizeof(arr));  // -> ? bytes
```

Integer в C

ИТМО

```c
printf("Size of 1: %ld bytes\n", sizeof(1));  // -> 4 bytes
printf("Size of 2 147 483 647: %ld bytes\n", sizeof(2147483647));  // -> 4 bytes
printf("Size of 2 147 483 648: %ld bytes\n", sizeof(2147483648));  // -> 8 bytes

printf("Size of int8_t: %ld bytes\n", sizeof(int8_t));  // -> 1 bytes
printf("Size of uint8_t: %ld bytes\n", sizeof(uint8_t));  // -> 1 bytes
printf("Size of char: %ld bytes\n", sizeof(char));  // -> 1 bytes
printf("Size of int s: %ld bytes\n", sizeof(short int));  // -> 2 bytes
printf("Size of int: %ld bytes\n", sizeof(int));  // -> 4 bytes
printf("Size of int l: %ld bytes\n", sizeof(long int));  // -> 8 bytes
printf("Size of int ll: %ld bytes\n", sizeof(long long));  // -> 8 bytes

int arr[10];
printf("Size of arr[10]: %ld bytes\n", sizeof(arr));  // -> 40 bytes
```

1 байт - $2^8 \in [0, 255] \mid [-128; 127]$

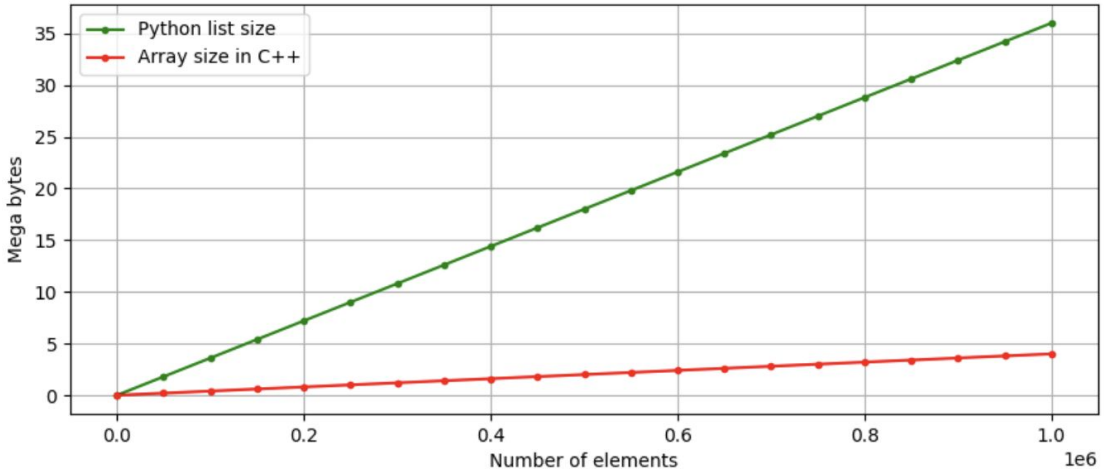
8 байт - ?

Integer в Python

ИТМО

```python
print(f'Size of 1: {getsizeof(1)}')  # 28 bytes
print(f'Size of 2 147 483 647: {getsizeof(2_147_483_647)}')  # 32 bytes
print(f'Size of 2 147 483 648: {getsizeof(2_147_483_648)}')  # 32 bytes

print(f'Size of 1e+21: {getsizeof(int(1e+21))}')  # 36 bytes
print(f'Size of 1e+36: {getsizeof(int(1e+36))}')  # 40 bytes
```



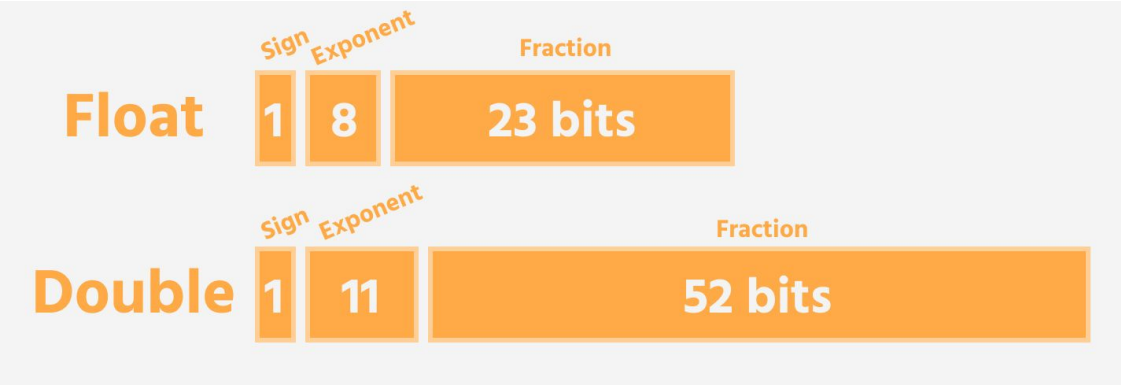
В Python массив на миллион int чисел весит в **5 раз больше**, чем в C (в теории)

Float и double

Компиляторы C используют стандарт **IEEE 754** (в основном):

- `float` → 32 бита (4 байта, от ~10^{-38} до ~10^{38}) =
  1 бит знак + 8 бит экспонента + 23 бита мантисса

- `double` → 64 бита (8 байт, от ~10^{-308} до ~10^{308}) =
  1 бит знак + 11 бит экспонента + 52 бита мантисса



Для 100 (`float`):
- sign = 0 → положительное
- exponent = 133 → 133 − 127 = 6
- fraction = 100100… (то есть 0.100100_2 = 0.5625)
- value = (1 + 0.5625) × 2^6 = 1.5625 × 64 = 100

Для 1010.1097 (`float`):
- sign = 0 → положительное
- exponent = 136 → 136 − 127 = 9
- fraction = 11111001000011100
- value = (1 + 0.111110010000111) × 2^9 = = 1010.109680

https://codefinity.com/blog/Float-vs-Double

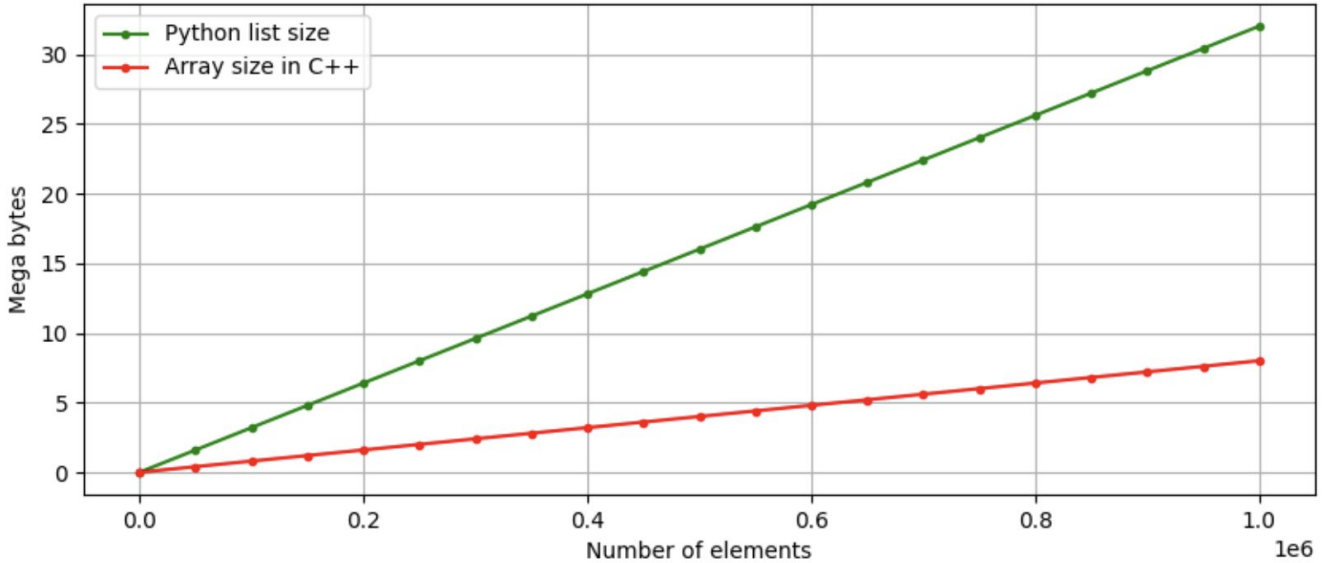
Float and Double в Python

ИТМО

```python
print(f'Size of 0.1: {getsizeof(0.1)}')  # Size of 0.1: 24

max_float = float_info.max
print(f'Size of {max_float}: {getsizeof(max_float)}')  # Size of 1.7976931348623157e+308: 24

more_than_max_float = float_info.max + 2
print(f'Size of {more_than_max_float}: {getsizeof(more_than_max_float)}')  # Size of 1.7976931348623157e+308: 24
```



В Python массив на миллион double чисел весит в **4 раз больше**, чем в C

Интервалы int значений

Тип	Размер в байтах (бит)	Интервал изменения
char	1 (8)	-128 до 127
unsigned char	1 (8)	от 0 до 255
int	4 (32)	от -2_147_483_648 до 2_147_483_647
unsigned int	4 (32)	от 0 до 4_294_967_295
short int	2 (16)	от -32768 до 32767
unsigned short int	2 (16)	от 0 до 65535
long int	8 (64)	от -2^63 до 2^63 - 1
unsigned long int	8 (64)	от 0 до 2^64 - 1
float	4 (32)	от -3.4028235e+38 до 3.4028235e+38
double	8 (64)	от -1.7976…e+308 до 1.7976…e+308
long double	16 (128)	-1.1897314…e+4932 до 1.1897314…e+4932

Char and String в C

ИТМО

```c
printf("Size of 'a': %ld bytes\n", sizeof('a'));  // -> ? bytes
printf("Size of \"a\": %ld bytes\n", sizeof("a"));  // -> ? bytes
printf("Size of 'ы': %ld bytes\n", sizeof(L'ы'));  // -> ? bytes
printf("Size of \"ы\": %ld bytes\n", sizeof("ы"));  // -> ? bytes

char char_ = 'a';
printf("Size of char 'a': %ld bytes\n", sizeof(char_));  // -> ? bytes
printf("Size of char: %ld bytes\n", sizeof(char));  // -> ? bytes

char str[] = "hello", str_k[] = "Привет";
printf("Size of 'hello': %ld bytes\n", sizeof(str));  // -> ? bytes
printf("Size of 'Привет': %ld bytes\n", sizeof(str_k));  // -> ? bytes
```

Char and String в C

ИТМО

```c
printf("Size of 'a': %ld bytes\n", sizeof('a'));  // -> 4 bytes
printf("Size of \"a\": %ld bytes\n", sizeof("a"));  // -> 2 bytes
printf("Size of 'ы': %ld bytes\n", sizeof(L'ы'));  // -> 4 bytes
printf("Size of \"ы\": %ld bytes\n", sizeof("ы"));  // -> 3 bytes

char char_ = 'a';
printf("Size of char 'a': %ld bytes\n", sizeof(char_));  // -> 1 bytes
printf("Size of char: %ld bytes\n", sizeof(char));  // -> 1 bytes

char str[] = "hello", str_k[] = "Привет";
printf("Size of 'hello': %ld bytes\n", sizeof(str));  // -> 6 bytes
printf("Size of 'Привет': %ld bytes\n", sizeof(str_k));  // -> 13
bytes
```

Char and String в Python

ИТМО

```
print(f'Size of "": {sys.getsizeof("")}')  # Size of "": 41
print(f'Size of "w": {sys.getsizeof("w")}')  # Size of "w": 42
print(f'Size of "ы": {sys.getsizeof("ы")}')  # Size of "ы": 60
print(f'Size of "ww": {sys.getsizeof("ww")}')  # Size of "ww": 43
print(f'Size of "цы": {sys.getsizeof("цы")}')  # Size of "цы": 62
print(f'Size of "wы": {sys.getsizeof("wы")}')  # Size of "wы": 62
```

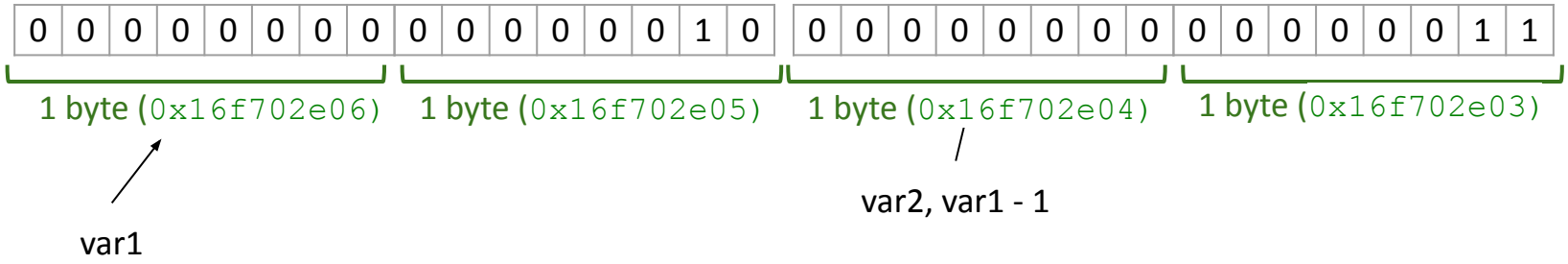
```
print(f'Size of "hello": {asizeof.asizeof("hello")}')  # Size of []: 48
print(f'Size of "Hello wor": {asizeof.asizeof("Hello wor")}')  # Size of []: 56
print(f'Size of "Hello world!": {asizeof.asizeof("Hello world!")}')  # Size of []: 56
print(f'Size of []: {asizeof.asizeof([])}')  # Size of []: 56
print(f'Size of ["hello", "world"]: {asizeof.asizeof(["hello", "world"])}')  # Size: 168
```

Указатели и ссылки

```
short int val1 = 1, val2 = 2;       // Создали переменную
short int* var1 = &val1;            // Создали указатель на 1
short int* var2 = &val2;            // Создали указатель на 2
printf("Size of var1: %ld bytes\n", sizeof(var1));  // -> 8 b
printf("Size of &val1: %ld bytes\n", sizeof(&val1));  // -> 8 b

// стек - от старших к младшим
printf("address 1: %p\n", (void*)var1);          // 0x16f702e06
printf("address 1: %p\n", (void*)&val1);         // 0x16f702e06
printf("address 2: %p\n", (void*)var2);          // 0x16f702e04
printf("address -1: %p\n", (void*)(var1 - 1));   // 0x16f702e04
```





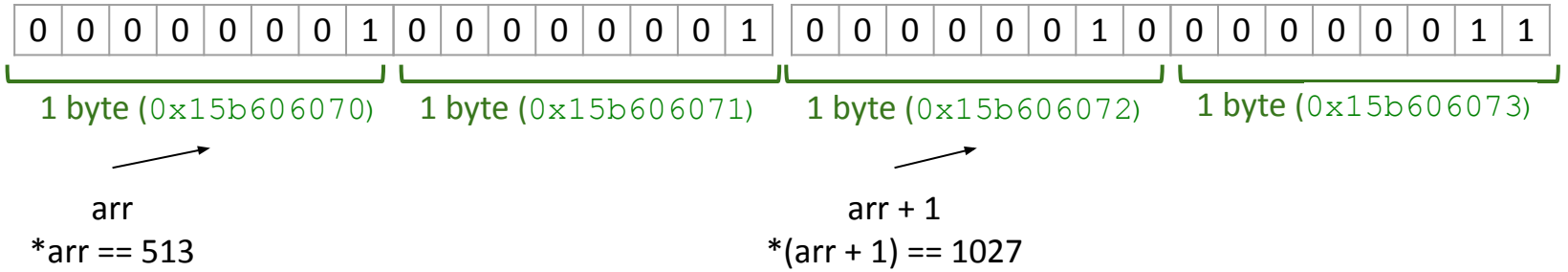
Указатели и динамическая память

ИТМО

```
short int* arr = malloc(2 * sizeof(short int));

printf("addres 1: %p\n", (void*)arr);          // 0x15b606070
printf("addres 2: %p\n", (void*)(arr + 1));    // 0x15b606072

printf("val1 in array: %d\n", *arr);           // 0
printf("val1 in array: %d\n", *(arr + 1));     // 0
free(arr);
```



Задание чтобы потренить перед 3-й лабой

ИТМО

- **Базовые (1–5)**

1. Объявить переменную int, создать указатель на неё, вывести адрес и значение через указатель.
2. Обменять значения двух переменных с помощью указателей.
3. Написать функцию, которая принимает int* и увеличивает значение на 1.
4. Создать массив int[5], пройти его с помощью указателя и вывести элементы.
5. Реализовать собственную функцию strlen с использованием указателей.

- **Средние (6–10)**

6. Реализовать функцию swap для двух элементов массива через указатели.
7. Написать функцию, которая копирует строку (char* src → char* dst) с помощью указателей.
8. Написать функцию, которая находит минимальный элемент массива с помощью указателя.
9. Вывести строку задом наперёд, двигаясь указателем от конца к началу.
10. Создать массив указателей на строки и вывести все строки.

- **Продвинутые (11–15)**

11. Реализовать функцию, которая принимает int** и выделяет динамический массив внутри (эмуляция «возврата массива»).
12. Создать двумерный массив динамически (int**), заполнить и вывести.
13. Написать функцию, которая принимает массив строк (char**) и сортирует их.
14. Реализовать функцию, которая ищет подстроку в строке (аналог strstr) через указатели.
15. Создать структуру с полем-указателем, выделить динамически память под поле и освободить её.

Структуры

ИТМО

```c
struct A { int x; };
struct B { char c; int x; };
struct C { int x; char c; };
struct D { char c; double d; };

printf("A: %zu\n", sizeof(struct A));  // 4 bytes
printf("B: %zu\n", sizeof(struct B));  // 8 bytes
printf("C: %zu\n", sizeof(struct C));  // 8 bytes
printf("D: %zu\n", sizeof(struct D));  // 16 bytes
```

Пример структуры для вектора

ИТМО

```
typedef struct {
    void *data;
    size_t size;
    size_t elem_size;
    size_t capacity;
} Vector;

Vector* vectorCreate(size_t elem_size);
```



Мне кажется

Или у вас есть вопросы?

ITMO more than a UNIVERSITY

Ivan.Khodnenko@itmo.ru